

Local DATV Laboratory

Pluto TX + local Pluto FFT + MiniTiouner decoding

Detailed implementation plan for QO100-RASPI-RECEIVER

Item	Reference
Receiver application	https://github.com/HB9IIU/QO100-RASPI-RECEIVER
PlutoDVB2 firmware	https://github.com/F5OEO/plutosdr-fw
Firmware application revision	pluto-ori-ps c8a47400b86018a970c0df6e3825a9a99c7d4622
Plan edition	1.0 - 5 September 2026

Goal

Create a standalone local over-the-air DATV test station that transmits at the real QO-100 uplink frequency, decodes with a MiniTiouner, displays a local Pluto FFT, and can optionally overlay the local waveform onto the live QO-100 WB spectrum.

1. Project objective

The laboratory reproduces the operator workflow of a QO-100 DATV station without sending a signal through the satellite. The RF link remains real: a small antenna on the Pluto TX port radiates a low-level 2.4 GHz signal and a second small antenna feeds the MiniTiouner.

Function	Device
DVB-S/S2 modulation and 2.4 GHz transmission	ADALM-Pluto running PlutoDVB2
Local spectrum acquisition	The RX channel of the same Pluto
FFT calculation and publication	PlutoDVB2 internal FFT service
DVB-S/S2 lock and decoding	MiniTiouner controlled by Longmynd
Spectrum, touch tuning, video and UI	Raspberry Pi running QO100-RASPI-RECEIVER
Optional real QO-100 background spectrum	BATC WB FFT WebSocket

Physical RF arrangement

Pluto TX port -> small 2.4 GHz antenna
short over-the-air path
MiniTiouner input <- small receiving antenna

Pluto RX port -> separate small RX antenna for the local FFT

There is no RF cable between the Pluto transmitter and MiniTiouner. The setup is a low-power local wireless DATV link.

2. Three operating modes

Mode	Spectrum source and tuning behavior
Satellite	Existing behavior: BATC live WB spectrum, 10 GHz labels, LNB-LO conversion, MiniTiouner tuned to LNB IF.
Local	Pluto local FFT, 2.4 GHz labels, no LNB conversion, MiniTiouner tuned directly to the selected uplink frequency.
Overlay	BATC live WB spectrum remains the background. The local Pluto waveform is mapped to its equivalent satellite downlink position and drawn above it.

The existing Satellite mode should remain unchanged. Local and Overlay are additions, selected from the settings page or a compact main-screen mode control.

3. Frequency model

The local system operates on QO-100 WB uplink-channel frequencies. Recommended channel centres currently run from 2403.250 MHz through 2409.750 MHz; the exact usable occupied bandwidth depends on symbol rate and the official channel plan.

Uplink/downlink mapping

```
simulated_downlink_MHz = local_uplink_MHz + 8089.500
```

Local uplink	Overlay position
2403.250 MHz	10492.750 MHz
2405.000 MHz	10494.500 MHz
2405.250 MHz	10494.750 MHz
2409.750 MHz	10499.250 MHz

Display scale

Keep a 9 MHz display span in every mode so that equal symbol rates occupy equal screen widths. The current application uses 10490.5-10499.5 MHz. A convenient local display is 2401.0-2410.0 MHz or 2401.5-2410.5 MHz; the final choice should align exactly with the desired frequency-label mapping.

Quantity	Target
Displayed width	9.000 MHz
Application display bins	2048 normalized bins
Nominal bin spacing	9 MHz / 2048 = 4.3945 kHz
Horizontal mapping	$x = (\text{frequency} - \text{display_start}) / 9 \text{ MHz} \times \text{plot_width}$

4. What PlutoDVB2 already provides

No FFT calculation needs to be added to the Python transmitter application. PlutoDVB2 already captures RX I/Q samples, applies a Hann window, calculates FFT power, joins sweep segments where required, and publishes binary FFT frames.

Interface	Verified behavior
MQTT control	TCP port 1883; command root cmd/pluto/<CALL>/...
FFT WebSocket	ws://<PLUTO-IP>:7681/websocket
Frame format	Binary array of unsigned 16-bit values in the Pluto CPU's native little-endian byte order.
FFT core size	2048 complex samples; the firmware publishes 1024 retained bins per sweep segment.
Wide span	PrepareSpan() can combine as many as eight RX fast-lock sweep segments.
Update interval	The source aims at roughly one output cycle every 40 ms, subject to sweep and processing load.

The number of received local bins may therefore differ from the BATC feed. The receiver application must resample the Pluto result onto its standard 2048-bin, 9 MHz display grid.

5. Pluto configuration

The Python DATV TX application needs only a small MQTT control component. It should not calculate or forward FFT data.

Startup command sequence

```
R00T=cmd/pluto/<CALL>

$R00T/rx/stream/run -> 1
$R00T/rx/webfft/frequency -> 2405750000 # example centre
$R00T/rx/webfft/span -> 9000000
$R00T/rx/stream/mode -> webfft
```

Recommended Python responsibility

Action	When
Connect to MQTT broker	At application startup, with reconnection enabled.
Publish FFT centre/span/mode	After MQTT connects and whenever Local/Overlay settings change.
Request status with '?'	After configuration, to confirm the effective frequency and span.
Disable or return RX to pass	Optionally when leaving Local/Overlay mode.
Do not transport FFT frames	The receiver application connects directly to port 7681.

Because PlutoDVB2 may adjust the requested span to an integer number of sweep segments, the UI should use the effective span reported on dt/pluto/<CALL>/rx/webfft/span rather than assuming that the request was accepted exactly.

6. Receiver-application architecture

Refactor the existing SpectrumFeed concept into interchangeable sources while leaving SpectrumTexture, signal detection and drawing as shared consumers.

Component	Responsibility
ISpectrumFeed	Common start/stop/status/frame interface.
BatcSpectrumFeed	Existing TLS WebSocket at eshail.batc.org.uk/wb/fft.
PlutoSpectrumFeed	Local plain WebSocket at Pluto port 7681, path /websocket.
SpectrumNormalizer	Noise-floor removal, amplitude scaling, smoothing and bin resampling.
SpectrumComposer	Select BATC, local, or BATC plus local overlay.
FrequencyModel	Display labels, uplink/downlink conversion and MiniTiouner tuning target.

Threads

Use one worker thread for each active WebSocket source. In Overlay mode, BATC and Pluto feeds run concurrently. Each worker should publish only its latest complete frame into a mutex-protected or lock-free handoff slot, matching the current BATC latest-frame design. Rendering remains on the SDL thread.

7. Local FFT processing

Pluto FFT values and BATC values are not guaranteed to share the same numerical calibration. Processing should be explicit and testable.

Stage	Operation
1. Decode	Validate an even byte count and decode little-endian uint16 bins.
2. Validate	Reject empty, implausibly short, stale or malformed frames.
3. Frequency grid	Use the effective Pluto FFT centre and span returned through MQTT.
4. Baseline	Estimate local noise with a robust percentile or median, not the absolute minimum.
5. Extract	Compute positive signal excess above the local baseline.
6. Resample	Interpolate onto the common 2048-bin, 9 MHz display grid.
7. Smooth	Apply modest temporal smoothing; keep enough response to show modulation changes.
8. Detect	Run the existing width/symbol-rate detector on normalized display bins.

Avoid copying the current fixed BATC constants directly into the Pluto decoder. Keep separate source decoders and normalize only at the common-display boundary.

8. Overlay and beacon normalization

Overlay mode should keep the live BATC spectrum untouched and add only the local signal component above the Pluto noise floor.

Core calculation

```
local_excess[i] = max(0, local[i] - local_noise)

scale = target_beacon_height / max(local_excess)

overlay[i] = local_excess[i] * scale

composite[i] = max(batc[i], batc_noise + overlay[i])
```

Control	Meaning
Beacon	Normalize the local peak to the measured BATC WB beacon peak.
-1 dB / -3 dB / -6 dB	Scale relative to beacon amplitude for a more conservative simulation.
Opacity	Optional visual blending; does not alter detection calculations.
LOCAL SIMULATION label	Makes it impossible to confuse the overlay with an actual satellite transmission.

Beacon measurement

Measure the beacon peak and nearby BATC baseline from fixed frequency windows. Smooth the result over several frames so that the overlay height does not jump. If the BATC feed or beacon is unavailable, freeze the last valid calibration briefly, then hide the overlay or show an explicit uncalibrated state.

The overlay predicts frequency occupancy, bandwidth and spectral shape. It does not predict the true satellite downlink level, MER or link margin.

9. Touch tuning and MiniTiouner

Mode	Touch-to-tune calculation
Satellite	Keep existing calculation: MiniTiouner IF = selected downlink - configured LNB LO.
Local	MiniTiouner frequency = selected 2.4 GHz frequency directly; LNB LO and bias controls are bypassed.
Overlay	A click on the simulated local signal maps downlink back to uplink: uplink = displayed downlink - 8089.5 MHz, then tunes MiniTiouner directly.

Do not implement Local mode by setting the existing LNB offset to zero: the current settings UI constrains the LO to 1000-20000 MHz. A dedicated frequency model is clearer and avoids accidental changes to Satellite mode.

MiniTiouner validation

Before software integration, confirm reliable lock across the desired 2403.250-2409.750 MHz channel centres and all intended symbol rates. Record any tuner sensitivity change near the top of its supported range.

10. RF safety and stability

Risk	Mitigation
Pluto RX overload	Use minimum practical TX gain, antenna separation and/or shielding. Watch the FFT for compression and raised broadband floor.
MiniTiouner overload	Increase antenna spacing or add attenuation before the tuner if required.
Radiating outside the laboratory	Use very low power, small antennas, short distance and preferably a screened enclosure or dummy-load/coupler option during development.
TX leakage masks shoulders	Increase isolation between Pluto TX and RX antennas; compare at several distances and orientations.
Frequency error	Use a stable Pluto reference if precise channel alignment is required.
Network loss	Reconnect MQTT and both WebSockets independently; never freeze the SDL UI while reconnecting.

The FFT is useful only while the Pluto receiver remains in its linear range. A perfectly tall trace produced by overload is not a valid measurement of spectral quality.

11. Implementation phases

Phase	Deliverable and acceptance criterion
1. Bench validation	Pluto transmits over air; MiniTiouner locks and decodes at several frequencies and symbol rates.
2. Pluto FFT proof	A small test client receives valid binary frames from ws://PLUTO:7681/websocket while TX remains active.
3. MQTT control	Python TX app enables WebFFT and reads back effective centre/span without blocking its UI.
4. Local source	Receiver app displays the Pluto FFT on a 9 MHz scale with correct frequency labels.
5. Direct tuning	Touching a detected local signal tunes MiniTiouner directly and produces video.
6. Calibration	333 kS and 500 kS signals have the same displayed width in Local and Satellite modes.
7. Overlay	Local excess spectrum maps by +8089.5 MHz and is visibly marked as simulated.
8. Beacon scaling	Overlay peak tracks Beacon, -1, -3 and -6 dB targets without jitter.
9. Resilience	Disconnecting Pluto, BATC internet or MiniTiouner does not freeze the interface; each path recovers independently.
10. Documentation	Settings, RF precautions and mode behavior are documented with screenshots.

12. Recommended first experiment

Before modifying the main receiver application, prove the key unknown with a minimal standalone client:

Step	Action
1	Install and boot the verified PlutoDVB2 firmware.
2	Configure a low-power DVB-S2 transmission and confirm MiniTiouner decoding.
3	Publish the four RX WebFFT MQTT settings.
4	Connect to ws://PLUTO-IP:7681/websocket.
5	Log frame byte lengths and decode uint16 values.
6	Plot one frame and verify that the transmitted carrier appears at the correct frequency.
7	Repeat while changing symbol rate to confirm measured-width behavior.

This proof should be completed before UI work. It confirms simultaneous TX/RX behavior, actual span, bin ordering, byte order, amplitude stability and required RF isolation.

13. Proposed settings

Setting	Suggested values
Spectrum Mode	Satellite / Local / Overlay
Pluto IP	Default 192.168.2.1
Pluto Callsign	Used in the MQTT command root
Local FFT Centre	Default 2405.750 MHz
Local FFT Requested Span	9.000 MHz
Overlay Level	Beacon / -1 dB / -3 dB / -6 dB
Overlay Colour	A distinct colour not used by the BATC gradient
MiniTiouner Local Mode	Direct 2.4 GHz tuning; LNB bias off

14. Definition of done

The project is complete when the operator can transmit locally, see the waveform at the correct scale, touch it to tune, decode it with MiniTiouner, and optionally preview how it would occupy the live QO-100 WB spectrum - all without altering the existing satellite-reception workflow.

15. Verified source references

Reference	URL / revision
Receiver application	https://github.com/HB9IIU/QO100-RASPI-RECEIVER , inspected commit 0078e5d43637a3fa62ff0978cdf2afbaff7ed3f8
Firmware project	https://github.com/F5OEO/plutosdr-fw
MQTT/FFT application	https://github.com/F5OEO/pluto-ori-ps/tree/c8a47400b86018a970c0df6e3825a9a99c7d4622
QO-100 WB plan	https://wiki.batc.org.uk/QO-100_WB_Bandplan

End of plan